Desarrollo de Aplicación Híbridas

UT4: Navegación, Servicios y Persistencia de Datos

UT4: Navegación, Servicios y Persistencia de Datos

¡Bienvenido/a a la Unidad de Trabajo 4! Hasta ahora, hemos construido componentes, les hemos dado estilo y hemos creado formularios, pero nuestra aplicación sigue teniendo dos limitaciones importantes:

1. Todo sucede en **una sola página**.
2. Todos los datos que añadimos (como una nueva tarea) **se borran** en cuanto recargamos la aplicación.

En esta unidad, vamos a solucionar estos dos problemas y convertiremos nuestro proyecto en una aplicación mucho más real, estructurada y profesional. Esta es, probablemente, la unidad más importante del módulo.

Dividiremos el trabajo en tres grandes bloques:

- **Servicios:** Aprenderemos a sacar la lógica (como la lista de tareas) de nuestros componentes y a centralizarla en "servicios", que son almacenes de lógica y datos que toda la app puede compartir.
- **Navegación:** Convertiremos nuestra app en una "Single Page Application" (SPA) real. Aprenderemos a usar el Enrutador de Angular para crear nuevas páginas (como "Detalle de Tarea") y a navegar entre ellas, pasando parámetros por la URL (ej: `/tareas/1`).
- **Persistencia:** Aprenderemos a guardar datos de forma permanente en el dispositivo usando **Ionic Storage**. Nos centraremos en guardar las preferencias del usuario (como el modo oscuro o su nombre), sentando las bases para la gestión de datos de una API real en la siguiente unidad.

Objetivos de Aprendizaje

Al finalizar esta unidad, serás capaz de:

- Comprender el concepto de Servicio en Angular y su importancia.

- Crear un servicio para centralizar la lógica de negocio (nuestro `TaskService`).
- Utilizar la Inyección de Dependencias para usar servicios dentro de los componentes.
- Comprender el sistema de enrutamiento (routing) de Angular y el archivo `app.routes.ts`.
- Navegar entre páginas usando `routerLink` (en el HTML) y el servicio `Router` (en TypeScript).
- Pasar datos (parámetros) a una página a través de la URL y leerlos con `ActivatedRoute`.
- Instalar, configurar y usar `Ionic Storage` para guardar datos de forma persistente.
- Crear un servicio de ajustes (`SettingsService`) para guardar y cargar las preferencias del usuario.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.1. El Problema de la Lógica en los Componentes

4.1. El Problema de la Lógica en los Componentes

En las unidades anteriores, para que nuestra aplicación de lista de tareas funcionara, hicimos esto en nuestro archivo `home.page.ts`:

```
// Contenido de: src/app/home/home.page.ts

// ...
export class HomePage {

  // ¡AQUÍ ESTÁ EL PROBLEMA!
  public listaDeTareas: Tarea[] = [
    { id: 1, titulo: "Comprar el pan", completada: true },
    { id: 2, titulo: "Estudiar componentes", completada: false }
  ];

  constructor() {}

  agregarTarea() {
    // ... lógica para añadir una tarea al array 'listaDeTareas'
  }
}
```

Esto funciona, pero es una **mala práctica de arquitectura de software** que nos traerá muchos problemas a medida que la aplicación crezca. El componente `HomePage` está haciendo dos trabajos a la vez: está controlando la **vista** (lo que se ve) y también está gestionando la **lógica de negocio y los datos** (es el dueño de la lista de tareas).

Esto viola un principio de diseño fundamental llamado **Principio de Responsabilidad Única** (un componente solo debería tener una razón para cambiar).

El Problema Principal: Los Datos están "Atrapados"

Imagina que en esta unidad queremos crear una nueva página llamada `DetalleTareaPage`. El objetivo de esta página será mostrar los detalles de una única tarea cuando el usuario haga clic en ella desde la lista.

Aquí nos encontramos con una pregunta imposible de responder con nuestra arquitectura actual:

> **Si la `listaDeTareas` vive dentro de `HomePage`, ¿cómo puede la `DetalleTareaPage` acceder a esa lista para buscar y mostrar la tarea con el ID 3?**

La respuesta es: no puede, o al menos no de una forma limpia. Los componentes en Angular están aislados. `HomePage` no sabe que `DetalleTareaPage` existe, y viceversa. Los datos están "atrapados" o "secuestrados" dentro del componente `HomePage`.

La Solución: Centralizar la Lógica en un Servicio

Para solucionar esto, necesitamos sacar los datos y la lógica de negocio (el array `listaDeTareas`, los métodos `agregarTarea`, `eliminarTarea`, etc.) fuera del componente.

Necesitamos crear un "almacén" o "cerebro" centralizado que no sea un componente visual. Un lugar al que cualquier componente de nuestra aplicación (`HomePage`, `DetalleTareaPage`, o cualquier otra) pueda "llamar" para pedirle los datos o para solicitarle que realice una acción.

Esa "caja de lógica" centralizada es lo que en Angular se conoce como un **Servicio**, y es lo que aprenderemos a crear en la siguiente sección.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.2. Servicios en Angular: El Principio de Responsabilidad Única

4.2. Servicios en Angular: El Principio de Responsabilidad Única

Como vimos en la sección anterior, nuestro problema es que los datos y la lógica de negocio (el array de tareas) están "atrapados" dentro de un componente visual (`HomePage`). La solución de Angular para este problema es el concepto de **Servicio**.

¿Qué es un Servicio?

Un servicio es una clase de TypeScript simple con un propósito muy específico: **contener lógica de negocio o datos que deben ser compartidos** por diferentes partes de tu aplicación. Un servicio no es un componente, no tiene plantilla HTML ni estilos. Es un "cerebro" en segundo plano, un simple archivo `.ts`.

Un servicio es el lugar perfecto para poner:

- El array `listaDeTareas`.
- El método `agregarTarea()`.
- Un futuro método `eliminarTarea()`.
- Un futuro método `getTareaPorId()` (que necesitará la página de detalle).

La Inyección de Dependencias (DI)

Los servicios se conectan a los componentes mediante un patrón de diseño llamado **Inyección de Dependencias (DI)**, que ya introdujimos en la unidad anterior. Es un concepto clave de Angular:

1. El componente (ej: `HomePage`) no crea el servicio.
2. El componente simplemente "pide" el servicio en su `constructor`.
3. Angular, que actúa como un "proveedor" centralizado, ve la petición y le entrega (le "inyecta") al componente la instancia del servicio que necesita.

La Magia de los Servicios: Son "Singletons"

Por defecto, Angular trata los servicios como **Singletons**. Esto es un concepto de programación que suena complicado pero es muy simple: significa que, aunque 10 componentes diferentes pidan el mismo servicio (ej: `TaskService`), Angular no crea 10 copias. Angular crea **una única instancia** de ese servicio y se la entrega a todos los componentes que la pidan.

Esto es exactamente lo que necesitamos. Si `HomePage` y `DetalleTareaPage` piden el `TaskService`, ambos recibirán **la misma instancia**, y por lo tanto, ambos tendrán acceso al **mismo array `listaDeTareas`**.

Así es como solucionamos el problema de los "datos atrapados":

> ### Definición: ¿Qué es un Singleton?
>
> El "Singleton" (o Instancia Única) es uno de los patrones de diseño de software más conocidos. Su objetivo es simple: **garantizar que una clase tenga una única instancia en toda la aplicación** y proporcionar un punto de acceso global a dicha instancia.
>
> En Angular, los servicios proporcionados "en el root" (`providedIn: 'root'`) son singletons. Esto es perfecto para gestionar un "estado" o una fuente de datos compartida, como nuestra lista de tareas, porque nos asegura que todos los componentes están mirando y modificando exactamente la misma información.

El Decorador `@Injectable`

Para que Angular sepa que una clase puede ser "inyectada" como un servicio, simplemente la marcamos con el decorador **`@Injectable`**. Cuando usamos la CLI para generar un servicio, este decorador ya se añade automáticamente.

```
// Esto es lo que veremos en un archivo de servicio

import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Esto le dice a Angular que el servicio es un singletor
})
export class MiServicio {
```

```
    constructor() { }
}
```

En la siguiente sección, pondremos esto en práctica. Crearemos nuestro `TaskService` y moveremos toda la lógica de las tareas a su nuevo hogar.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.3. Creando nuestro TaskService

4.3. Creando nuestro TaskService

Ha llegado el momento de aplicar la teoría. Vamos a crear un servicio llamado `TaskService` que actuará como el "almacén" centralizado para nuestra lista de tareas. Toda la lógica que antes estaba en `home.page.ts` (el array de datos, el método para añadir tareas) la moveremos aquí.

Paso 1: Generar el Servicio con la CLI

Al igual que generamos componentes o páginas, Ionic nos da un comando de la CLI para generar la estructura básica de un servicio. Es una buena práctica guardarlos en una carpeta propia llamada `services`.

Abre la terminal en la raíz de tu proyecto y ejecuta:

```
ionic generate service services/task.service
```

Este comando creará dos archivos:

- `src/app/services/task.service.ts`: Este es el archivo de nuestra clase de servicio (el que vamos a editar).
- `src/app/services/task.service.spec.ts`: Un archivo para pruebas (que por ahora ignoraremos).

Si abres el archivo `task.service.ts`, verás que ya viene con el decorador `@Injectable({ providedIn: 'root' })`, listo para funcionar como un singleton.

Paso 2: Mover la Lógica al Servicio

Ahora, vamos a editar el archivo `src/app/services/task.service.ts`. Cortaremos la lógica de `home.page.ts` y la pegaremos aquí, creando métodos públicos que nuestros componentes puedan llamar.

```
// Contenido de: src/app/services/task.service.ts

import { Injectable } from '@angular/core';
import { Tarea } from '../interfaces/tarea'; // Importamos nuestra interfaz

@Injectable({
  providedIn: 'root'
})
export class TaskService {

  // ¡La lógica de negocio vive aquí ahora!
  // Hacemos el array 'private' para que los componentes
  // no puedan modificarlo directamente, solo a través de nuestros métodos.
  private listaDeTareas: Tarea[] = [
    { id: 1, titulo: "Comprar el pan", completada: true },
    { id: 2, titulo: "Estudiar Servicios de Angular", completada: false }
  ];

  constructor() { }

  /**
   * Método público para obtener todas las tareas.
   * Devuelve una copia del array para proteger el original.
   */
  getTareas(): Tarea[] {
    return [...this.listaDeTareas]; // Usamos '...' (spread syntax) para devol
  }

  /**
   * Método público para añadir una nueva tarea.
   * Recibe el título de la nueva tarea como argumento.
   * @param titulo El título de la nueva tarea.
   */
  agregarTarea(titulo: string) {
    // Verificamos que el título no esté vacío
    if (titulo.trim().length === 0) {
      return; // No hacemos nada si está vacío
    }

    // Creamos la nueva tarea
```

```
    const nuevaTarea: Tarea = {
      id: Date.now(), // Usamos un timestamp como ID simple
      titulo: titulo,
      completada: false
    };

    // Añadimos la nueva tarea al principio de nuestro array
    this.listaDeTareas.unshift(nuevaTarea);
  }
}
```

Lo que hemos conseguido

¡Perfecto! Acabamos de crear un "cerebro" para nuestra aplicación. Fíjate en los cambios:

- `home.page.ts` (y cualquier otra página) ya no sabe de dónde vienen las tareas. Su única responsabilidad será **pedirle** las tareas al `TaskService`.
- La `listaDeTareas` es ahora `private`. Esto es una buena práctica de encapsulación. Obligamos a los componentes a usar nuestros métodos (`getTareas`, `agregarTarea`) y no pueden manipular el array directamente.
- El método `agregarTarea` ahora toma el `titulo` como un parámetro, haciendo que el servicio sea más reutilizable.

En la siguiente sección, veremos cómo "inyectar" y usar este nuevo servicio en nuestro componente `HomePage`.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.4. Inyectando y Usando el Servicio

4.4. Inyectando y Usando el Servicio

En la sección anterior, creamos nuestro `TaskService` y movimos toda la lógica de las tareas a él. Ahora, nuestro componente `home.page.ts` está obsoleto y necesita ser actualizado para "consumir" (usar) ese servicio.

Este proceso se llama **Inyección de Dependencias** y es el pilar de la arquitectura de Angular.

Paso 1: Inyectar el Servicio en `home.page.ts`

El primer paso es "pedirle" a Angular que nos entregue una instancia de nuestro `TaskService`. Hacemos esto en el `constructor` de la página.

```
// Contenido para añadir/modificar en: src/app/home/home.page.ts

import { Component } from '@angular/core';
// 1. Importamos nuestro nuevo servicio
import { TaskService } from '../services/task.service';
// ... otros imports ...

@Component({
  // ...
})
export class HomePage {

  // ...

  // 2. Inyectamos el servicio en el constructor
  // La palabra 'private' hace que 'taskService' se convierta
  // automáticamente en una propiedad de esta clase (this.taskService)
  constructor(private taskService: TaskService) {}
```

```
  // ...
}
```

¡Eso es todo! Con solo hacer esto, `HomePage` ya tiene acceso a la instancia única (singleton) de nuestro `TaskService` y a todos sus métodos públicos.

Paso 2: Refactorizar `home.page.ts` para Usar el Servicio

Ahora que tenemos el servicio, vamos a limpiar `home.page.ts` de toda la lógica que ya no le pertenece y la reemplazaremos por llamadas al servicio.

```
// Contenido COMPLETO de: src/app/home/home.page.ts

import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonList, IonItem, IonInp
import { Tarea } from '../interfaces/tarea';
import { TaskItemComponent } from '../components/task-item/task-item.component
import { CommonModule } from '@angular/common';
import { TaskService } from '../services/task.service'; // ¡Importante!

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  standalone: true,
  imports: [
    CommonModule, FormsModule, IonHeader, IonToolbar, IonTitle, IonContent,
    IonList, IonItem, IonInput, IonButton, TaskItemComponent, IonGrid, IonRow,
  ],
})
export class HomePage implements OnInit {

  // Ya no definimos la lista aquí.
  // Esta propiedad guardará la lista de tareas QUE NOS DE EL SERVICIO
  public listaDeTareas: Tarea[] = [];

  // Esta propiedad es para el formulario, como la teníamos
  public nuevaTareaTitulo: string = "";

  // Inyectamos el servicio
```

```
  constructor(private taskService: TaskService) {}

  // Usamos ngOnInit para cargar los datos iniciales
  ngOnInit() {
    this.cargarTareas();
  }

  /**
   * Pide las tareas al servicio y actualiza la lista local
   */
  cargarTareas() {
    this.listaDeTareas = this.taskService.getTareas();
  }

  /**
   * Método para añadir la tarea. Ahora, solo llama al servicio.
   */
  agregarTarea() {
    // Verificamos que el título no esté vacío
    if (this.nuevaTareaTitulo.trim().length === 0) {
      return;
    }

    // 1. Le pasamos el título al servicio. El servicio se encarga de crear la
    this.taskService.agregarTarea(this.nuevaTareaTitulo);

    // 2. Limpiamos el input
    this.nuevaTareaTitulo = "";

    // 3. Volvemos a pedir la lista actualizada al servicio para refrescar la
    this.cargarTareas();
  }
}
```

Fíjate en lo limpio que ha quedado el componente. `HomePage` ya no sabe *cómo* se crea o se guarda una tarea; su única responsabilidad es recoger el título del input y pasárselo al servicio.

Paso 3: Actualizar `home.page.html`

Ahora que nuestro `home.page.ts` usa la variable `nuevaTareaTitulo` (en lugar del objeto `nuevaTarea`), debemos actualizar el `[(ngModel)]` en nuestra plantilla HTML.

```
<!-- Contenido para modificar en: src/app/home/home.page.html -->

<!-- FORMULARIO PARA AÑADIR TAREAS -->
<ion-list>
  <ion-item>
    <ion-input
      label="Nueva Tarea"
      label-placement="floating"
      placeholder="¿Qué necesitas hacer?"
      [(ngModel)]="nuevaTareaTitulo"> <!-- CAMBIO AQUÍ: enlazamos a la variabl
    </ion-input>
  </ion-item>
</ion-list>

<ion-button expand="block" (click)="agregarTarea()" class="ion-margin-top ion-
  Añadir Tarea
</ion-button>
```

¡Y listo! La aplicación vuelve a funcionar exactamente igual que antes, pero ahora tenemos una arquitectura de código profesional, escalable y fácil de mantener.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.5. El Enrutador de Angular (app.routes.ts)

4.5. El Enrutador de Angular (app.routes.ts)

Una aplicación moderna de tipo SPA (Single Page Application) no recarga la página del navegador cada vez que el usuario cambia de pantalla. En su lugar, utiliza un sistema de **Enrutamiento (Routing)** para intercambiar componentes visuales dinámicamente, dando la ilusión de navegación.

En tu proyecto Ionic/Angular, hay un archivo que actúa como el "mapa de carreteras" de tu aplicación: **`src/app/app.routes.ts`**. Este archivo define qué componente se debe mostrar para cada URL.

Anatomía de una Ruta

Si abres `app.routes.ts`, verás un array llamado `routes`. Cada objeto dentro de ese array es una definición de ruta. Vamos a analizar la ruta que ya tienes creada por defecto para tu página de inicio:

```
// src/app/app.routes.ts

import { Routes } from '@angular/router';

export const routes: Routes = [
  // Definición de la ruta 'home'
  {
    path: 'home',
    loadComponent: () => import('./home/home.page').then((m) => m.HomePage),
  },
  // Ruta de redirección inicial (si la URL está vacía, va a home)
  {
    path: '',
    redirectTo: 'home',
```

```
    pathMatch: 'full',
  },
];
```

Las Propiedades Clave:

- **`path`**: Es el fragmento de la URL que activa esta ruta. Por ejemplo, si el path es `'home'`, esta ruta se activará cuando la URL del navegador sea `.../home`.
  - **Nota:** Un path vacío `''` representa la raíz de la aplicación (cuando entras a `localhost:8100/`).
- **`loadComponent`** (Lazy Loading): En lugar de importar el componente directamente al principio del archivo, usamos una función de importación dinámica (`import(...)`).
  - **¿Por qué?** Esto se llama **Carga Perezosa (Lazy Loading)**. Significa que el código de la página `HomePage` NO se descargará al navegador hasta que el usuario intente visitarla. Esto hace que la aplicación arranque mucho más rápido.
- **`redirectTo`**: Le dice al enrutador: "Si el usuario intenta entrar aquí (ej: la raíz `''`), envíalo automáticamente a esta otra ruta (ej: `'home'`)".
- **`pathMatch: 'full'`**: Es obligatorio cuando definimos redirecciones desde el path vacío. Asegura que la redirección solo ocurra si la URL es *exactamente* vacía, y no una que *empiece* por vacío (que son todas).

Cómo Añadir Nuevas Páginas

La buena noticia es que rara vez tendrás que escribir estas rutas a mano. Cuando utilizas la CLI de Ionic para generar una nueva página, ella se encarga de actualizar este archivo automáticamente.

Por ejemplo, si ejecutamos:

```
ionic generate page pages/detalle-tarea
```

La CLI creará los archivos de la página y añadirá automáticamente el siguiente objeto a tu array `routes`:

```
  {
    path: 'detalle-tarea',
```

```
    loadComponent: () => import('./pages/detalle-tarea/detalle-tarea.page').th
  },
```

Ahora tu aplicación ya sabe responder a la URL `.../detalle-tarea`. En la siguiente sección, veremos cómo poner enlaces en nuestro HTML para que el usuario pueda navegar hasta allí.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.6. Navegación Básica con routerLink

4.6. Navegación Básica con routerLink

Ya tenemos nuestras rutas definidas, pero el usuario no va a escribir las URLs manualmente en la barra del navegador. Necesitamos poner enlaces (links) en nuestra interfaz para que pueda moverse entre páginas.

En una web tradicional usaríamos la etiqueta `<a href="/destino">`. Sin embargo, en Angular **NO debemos usar `href`** para la navegación interna.

> ⚠️ **¿Por qué no usar `href`?**
>
> Si usas `href`, el navegador tratará el enlace como una navegación estándar: **recargará la página completa**. Esto reinicia tu aplicación, borra la memoria (adiós al estado de tus variables) y destruye la experiencia de "aplicación nativa" fluida que buscamos.

La Solución: La Directiva `routerLink`

Angular nos proporciona una directiva llamada **`routerLink`**. Cuando la usamos, le dice al navegador: "No recargues la página; simplemente cambia la URL y pídele a Angular que cargue el componente correspondiente".

Ejemplo Práctico: Creando una página "Acerca de"

Para probar la navegación, vamos a crear una página sencilla de información.

1. Generar la página

```
ionic generate page pages/about
```

Esto crea la página y añade automáticamente la ruta `'about'` en `app.routes.ts`.

2. Añadir el enlace en la Home

Vamos a `home.page.html` y añadimos un botón o un icono en la cabecera que nos lleve a esta nueva página. Fíjate en cómo usamos `routerLink` en lugar de `href`.

```
<!-- src/app/home/home.page.html -->

<ion-header>
  <ion-toolbar color="primary">
    <ion-title>Mis Tareas</ion-title>
    <ion-buttons slot="end">
      <!-- Este botón navega a la ruta '/about' -->
      <ion-button routerLink="/about">
        <ion-icon slot="icon-only" name="information-circle-outline"></ion-icc
      </ion-button>
    </ion-buttons>
  </ion-toolbar>
</ion-header>
```

(Recuerda registrar el icono `information-circle-outline` en tu `app.component.ts` si no lo has hecho).

Volver Atrás: El Botón de Regreso

Ahora podemos ir a la página "About", pero... ¿cómo volvemos? Ionic tiene un componente mágico para esto: **`<ion-back-button>`**.

Este componente es inteligente: solo aparece si hay un historial de navegación (es decir, si has llegado aquí desde otra página) y sabe automáticamente a dónde volver.

Vamos a editar la cabecera de nuestra nueva página `about.page.html`:

```
<!-- src/app/pages/about/about.page.html -->
```

```
<ion-header>
  <ion-toolbar color="primary">
    <ion-buttons slot="start">
      <!-- Este botón gestiona la navegación 'atrás' automáticamente -->
      <ion-back-button defaultHref="/home"></ion-back-button>
    </ion-buttons>
    <ion-title>Acerca de</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <h2>Gestor de Tareas v1.0</h2>
  <p>Aplicación desarrollada para el módulo de DAH.</p>
</ion-content>
```

Nota sobre `defaultHref`: Es una buena práctica usar esta propiedad. Le dice al botón dónde ir si el usuario ha aterrizado en esta página directamente (por ejemplo, escribiendo la URL o recargando el navegador) y no tiene historial previo.

Controlando la Animación: `routerDirection`

Ionic añade animaciones de transición nativas entre páginas (deslizamiento lateral en iOS, fade/rise en Android). Podemos ayudar a Ionic a saber qué animación usar con la propiedad `routerDirection`:

- `routerDirection="forward"`: (Por defecto) Animación de "ir hacia adelante" (entra nueva página).
- `routerDirection="back"`: Animación de "volver atrás" (la página actual se retira).
- `routerDirection="root"`: Animación de "cambio de raíz", útil para menús principales (sin animación de retroceso).

```
<ion-button routerLink="/about" routerDirection="forward">
  Ir a About
</ion-button>
```

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.7. Navegación Programática (El Servicio Router)

4.7. Navegación Programática (El Servicio Router)

La directiva `routerLink` es perfecta para enlaces estáticos, como un menú o un botón de "Atrás". Pero hay situaciones donde no podemos usarla porque la decisión de navegar depende del resultado de una operación lógica.

Imagina este escenario: El usuario rellena un formulario de login y pulsa "Entrar". Tu código primero tiene que validar las credenciales con un servidor y, **solo si son correctas**, navegar a la página principal. Esto no se puede hacer con un simple enlace en el HTML.

El Servicio `Router` de Angular

Para navegar desde nuestro código TypeScript, Angular nos proporciona el servicio **`Router`**. Como cualquier otro servicio, para usarlo primero debemos inyectarlo.

Pasos para la Navegación Programática

1. **Importar** `Router` desde `@angular/router`.
2. **Inyectar** el servicio en el constructor de tu clase.
3. **Llamar** al método `this.router.navigate(...)` cuando quieras cambiar de página.

Ejemplo Práctico

Vamos a modificar nuestra `HomePage`. Imagina que queremos que, al pulsar un botón, la aplicación haga un cálculo o guarde algo y, al terminar, nos lleve a la página "About".

1. La Lógica en `home.page.ts`

```
// src/app/home/home.page.ts
```

```
import { Component } from '@angular/core';
// 1. Importamos Router
import { Router } from '@angular/router';
// ... otros imports

@Component({
  // ...
})
export class HomePage {

  // 2. Inyectamos el Router en el constructor
  constructor(private router: Router) {}

  // Este método se ejecutará cuando pulsemos el botón
  navegarAAbout() {
    console.log("Realizando operaciones previas...");

    // Aquí iría tu lógica (ej: guardar datos, validar...)

    console.log("Navegando a la página About...");

    // 3. Usamos el método .navigate()
    // Acepta un array con los fragmentos de la ruta
    this.router.navigate(['/about']);
  }
}
```

2. La Vista en `home.page.html`

Ahora, en lugar de `routerLink`, usamos el evento `(click)` para llamar a nuestro método.

```
<!-- src/app/home/home.page.html -->

<ion-button (click)="navegarAAbout()">
  Ir a About (con lógica)
</ion-button>
```

Sintaxis del Método `navigate`

El método `navigate` espera un **array de comandos**. Esto es muy potente porque nos permite construir la URL por partes, lo cual será fundamental en la próxima sección cuando pasemos

parámetros.

- Para ir a `/home`: `this.router.navigate(['/home'])`
- Para ir a `/tareas/detalle`: `this.router.navigate(['/tareas', 'detalle'])`
- Para ir a `/tareas/15` (paramétrico): `this.router.navigate(['/tareas', 15])`

> ### Nota sobre `NavController` de Ionic
>
> Si lees documentación antigua o foros de Ionic, es posible que veas referencias a `NavController`. Es un servicio propio de Ionic que funciona de forma similar al `Router` de Angular pero añade mejor control sobre las animaciones de retroceso nativas.
>
> Sin embargo, en las versiones modernas de Ionic con Angular, **se recomienda usar el `Router` estándar de Angular** siempre que sea posible, ya que Ionic se integra con él automáticamente para gestionar las animaciones.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.8. Pasando Datos con Rutas (Parámetros)

4.8. Pasando Datos con Rutas (Parámetros)

Hasta ahora hemos navegado a rutas estáticas como `/home` o `/about`. Pero, ¿qué pasa si queremos ver el detalle de una tarea específica? No podemos crear una ruta diferente para cada tarea (`/tarea-1`, `/tarea-2`...).

Necesitamos una ruta dinámica que acepte un parámetro, algo así: **`/detalle-tarea/15`**, donde "15" es el ID de la tarea que queremos ver.

Paso 1: Configurar la Ruta con Parámetros

Para decirle al enrutador que una parte de la URL es variable, usamos los dos puntos `:` en la definición del `path`.

Vamos a modificar `app.routes.ts` (suponiendo que ya has generado la página `DetalleTareaPage` con `ionic g page pages/detalle-tarea`).

```
// src/app/app.routes.ts

export const routes: Routes = [
  // ...
  {
    // Añadimos '/:id' al final.
    // Esto significa: "Cualquier cosa que vaya después de 'detalle-tarea/' se
    path: 'detalle-tarea/:id',
    loadComponent: () => import('./pages/detalle-tarea/detalle-tarea.page').th
  },
];
```

Paso 2: Navegar Enviando el Parámetro

Ahora tenemos que actualizar nuestros enlaces para incluir el ID de la tarea.

⚠️ Requisito Obligatorio: Importar `RouterLink`

Para usar la directiva `[routerLink]` en tu HTML (especialmente en componentes standalone), **debes importarla** en tu archivo `home.page.ts`. Si no lo haces, obtendrás el error: *"Can't bind to 'routerLink' since it isn't a known property..."*.

```
// src/app/home/home.page.ts
import { RouterLink } from '@angular/router'; // 1. Importar

@Component({
  // ...
  imports: [
    // ... otros imports ...
    RouterLink // 2. Añadir al array de imports
  ]
})
export class HomePage ...
```

Opción A1: Lista Estándar (`ion-list`)

Si estás usando una lista simple de Ionic, la directiva `routerLink` se coloca en el `ion-item`. Pasamos un array donde el primer elemento es la ruta base y el segundo es el valor del parámetro.

```
<!-- Ejemplo con lista simple -->
<ion-list>
  <ion-item *ngFor="let tarea of listaDeTareas"
            [routerLink]="['/detalle-tarea', tarea.id]">
    <ion-label>{{ tarea.titulo }}</ion-label>
```

```
    </ion-item>
  </ion-list>
```

Opción A2: Rejilla con Componentes (`ion-grid`)

Si estás siguiendo la estructura de nuestro proyecto con una rejilla y el componente personalizado `app-task-item`, debes añadir la directiva `routerLink` directamente a la etiqueta de tu componente.

```
<!-- src/app/home/home.page.html -->

<ion-grid>
  <ion-row>
    <!-- El bucle *ngFor está en la columna -->
    <ion-col *ngFor="let tarea of listaDeTareas" size="12" size-md="6" size-lg

      <!-- AÑADIMOS EL ROUTERLINK AQUÍ -->
      <app-task-item
        [tarea]="tarea"
        [routerLink]="['/detalle-tarea', tarea.id]"
        style="cursor: pointer">
      </app-task-item>

    </ion-col>
  </ion-row>
</ion-grid>
```

(Nota: Hemos añadido `style="cursor: pointer"` para que al pasar el ratón por encima en un ordenador se vea la "manita", indicando que es un enlace).

Opción B: Usando `Router` (TypeScript)

Si navegas desde el código (por ejemplo, tras pulsar un botón que realiza una acción):

```
verDetalle(id: number) {
  this.router.navigate(['/detalle-tarea', id]);
}
```

Paso 3: Leer el Parámetro en la Página de Destino

Una vez en `DetalleTareaPage`, necesitamos saber qué ID nos han pasado para poder pedirle al `TaskService` los datos de esa tarea específica.

Nota importante sobre Imports: La CLI añade `FormsModule` automáticamente. Como esta página es solo de lectura (no tiene formularios), puedes borrarlo. Sin embargo, **debes añadir `RouterLink`** si vas a usar navegación en el HTML.

```
// src/app/pages/detalle-tarea/detalle-tarea.page.ts

import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, RouterLink } from '@angular/router'; // Importar Rout
import { TaskService } from 'src/app/services/task.service';
import { Tarea } from 'src/app/interfaces/tarea';
import { CommonModule } from '@angular/common';
import { IonHeader, IonToolbar, IonButtons, IonBackButton, IonTitle, IonConter

@Component({
  selector: 'app-detalle-tarea',
  templateUrl: './detalle-tarea.page.html',
  styleUrls: ['./detalle-tarea.page.scss'],
  standalone: true,
  imports: [
    CommonModule,
    RouterLink, // ¡Necesario para el routerLink en el HTML!
    IonHeader, IonToolbar, IonButtons, IonBackButton, IonTitle, IonContent, Ic
  ]
})
export class DetalleTareaPage implements OnInit {

  tarea: Tarea | undefined;

  constructor(
    private activatedRoute: ActivatedRoute, // Inyectar ActivatedRoute
    private taskService: TaskService        // Inyectar nuestro servicio de ta
  ) { }

  ngOnInit() {
    // Leer el parámetro 'id' de la URL
    const id = this.activatedRoute.snapshot.paramMap.get('id');

    if (id) {
```

```
      // Usar el ID para buscar la tarea
      this.tarea = this.taskService.getTareaPorId(Number(id));
    }
  }
}
```

> ### El método `getTareaPorId`
>
> Recuerda que tendrás que añadir un método en tu `TaskService` que reciba un ID y devuelva la tarea correspondiente:
>
> ```
> // En task.service.ts
> getTareaPorId(id: number): Tarea | undefined {
>   return this.listaDeTareas.find(t => t.id === id);
> }
> ```

Paso 4: Mostrar los Datos en la Vista (HTML)

Ahora que tenemos el objeto `tarea` cargado (o `undefined` si no se encontró), vamos a mostrarlo en la plantilla HTML de la página de detalle.

```
<!-- src/app/pages/detalle-tarea/detalle-tarea.page.html -->

<ion-header>
  <ion-toolbar>
    <ion-buttons slot="start">
      <!-- Botón para volver atrás automáticamente -->
      <ion-back-button defaultHref="/home"></ion-back-button>
    </ion-buttons>
    <ion-title>Detalle de Tarea</ion-title>
  </ion-toolbar>
</ion-header>
```

```
<ion-content class="ion-padding">

  <!-- Usamos *ngIf para comprobar si la tarea existe -->
  <div *ngIf="tarea; else noEncontrada">
    <h1>{{ tarea.titulo }}</h1>

    <p>
      Estado:
      <!-- Mostramos un badge de color según si está completada -->
      <ion-badge [color]="tarea.completada ? 'success' : 'warning'">
        {{ tarea.completada ? 'Completada' : 'Pendiente' }}
      </ion-badge>
    </p>

    <p><small>ID: {{ tarea.id }}</small></p>
  </div>

  <!-- Esta plantilla se muestra si 'tarea' es undefined (el 'else' del ngIf) -->
  <ng-template #noEncontrada>
    <div class="ion-text-center">
      <p>Tarea no encontrada.</p>
      <!-- Aquí usamos routerLink, por eso lo importamos en el TS -->
      <ion-button routerLink="/home">Volver al listado</ion-button>
    </div>
  </ng-template>

</ion-content>
```

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.9. El Problema de la Persistencia de Ajustes

4.9. El Problema de la Persistencia de Ajustes

Hasta ahora hemos avanzado mucho: tenemos una aplicación con arquitectura limpia (servicios) y capacidad de navegación (rutas y parámetros). Sin embargo, todavía sufrimos un problema fundamental que impide que nuestra aplicación se sienta "real": **la amnesia**.

El Problema: Todo se borra al recargar

Haz la prueba: abre tu aplicación, añade tres tareas nuevas y luego recarga la página del navegador (o cierra y abre la app en el móvil). ¿Qué ocurre? **Todo desaparece** y volvemos al estado inicial.

Esto ocurre porque, actualmente, nuestros datos (el array de tareas) viven únicamente en la **memoria RAM** del dispositivo. La memoria RAM es volátil; cuando la aplicación se cierra, el sistema operativo libera esa memoria y los datos se pierden para siempre.

¿Dónde se guardan los datos en una App Real?

Aquí es importante hacer una distinción de arquitectura profesional. En las aplicaciones modernas, manejamos dos tipos de persistencia con tecnologías muy diferentes:

> #### 1. Datos de Negocio (En la Nube / Backend)
>
> Los datos principales (tus tareas) se guardan en una **Base de Datos Externa** (como MySQL, MongoDB o Firebase) alojada en un servidor.
>
> Tu aplicación Ionic **NO** se conecta directamente a esa base de datos. En su lugar, utiliza **peticiones HTTP (API REST)** para comunicarse con el servidor. Esto es lo que implementaremos en la **UT5** usando `HttpClient`.

> **Nota sobre el Backend del Curso:** Para aprender a trabajar con APIs sin complicarnos configurando servidores reales, en este módulo utilizaremos **JSON Server**. Es una herramienta estándar que nos permitirá simular una API REST completa y funcional en cuestión de segundos.
>
> *Por eso, en esta unidad, las tareas seguirán borrándose al reiniciar. No queremos implementar un almacenamiento local que luego tendremos que borrar para conectarnos a la API real.*

2. Preferencias del Usuario (En el Dispositivo)

Hay datos que son específicos del dispositivo y no necesitan viajar a la nube, como el estado del **Modo Oscuro** o el nombre del usuario. Estos datos se guardan en el **Almacenamiento Local** del dispositivo.

El Nuevo Caso de Uso: Persistencia de Ajustes

Para aprender a usar el almacenamiento local sin trabajar en vano, vamos a implementar un sistema de preferencias que sí será definitivo. Haremos que el usuario pueda:

1. Elegir si prefiere el **Modo Oscuro** o el Modo Claro.
2. Guardar su **Nombre** para que la app le salude.

La Solución: Ionic Storage

Para guardar estos ajustes en el "disco duro" del móvil, utilizaremos **Ionic Storage**. Esta librería es un "camaleón" inteligente que elige la mejor tecnología de base de datos disponible según dónde se ejecute tu app:

- **En Móvil (Nativo):** Usa **SQLite**, una base de datos SQL robusta y rápida.
- **En Web / PWA:** Usa **IndexedDB**, la base de datos NoSQL integrada en el navegador.

Lo mejor es que tú usas el mismo código en ambos casos. En las siguientes secciones, aprenderemos a instalarla y configurarla.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.10. Solución: Ionic Storage

4.10. Solución: Ionic Storage

Para implementar nuestra persistencia de datos, vamos a utilizar la librería oficial **Ionic Storage**. Esta herramienta nos proporciona una forma sencilla de guardar datos en formato **Clave-Valor** (Key-Value).

Es decir, guardaremos la información asignándole una etiqueta única, por ejemplo:

- `"modo_oscuro" : true`
- `"nombre_usuario" : "Javier"`

Paso 1: Instalación

Ionic Storage no viene instalado por defecto en el proyecto. Necesitamos añadir la librería de Angular y la librería base de almacenamiento (que contiene los drivers).

Abre la terminal en la carpeta de tu proyecto y ejecuta el siguiente comando corregido:

```
npm install @ionic/storage-angular @ionic/storage
```

Paso 2: Configuración en `main.ts` (Standalone)

Al igual que hicimos con el enrutador o los controladores de Ionic, necesitamos "registrar" el módulo de almacenamiento para que Angular sepa cómo usarlo en toda la aplicación.

En una aplicación **Standalone**, esto se hace en el archivo **`src/main.ts`**.

1. Abrimos `src/main.ts`.
2. Importamos `importProvidersFrom` de `@angular/core`.
3. Importamos `IonicStorageModule` de `@ionic/storage-angular`.
4. Añadimos la configuración dentro del array `providers`, manteniendo lo que ya tenías.

```
// src/main.ts

import { bootstrapApplication } from '@angular/platform-browser';
import { RouteReuseStrategy, provideRouter, withPreloading, PreloadAllModules
import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular/standa

// 1. Importamos estas utilidades
import { importProvidersFrom } from '@angular/core';
import { IonicStorageModule } from '@ionic/storage-angular';
import { Drivers } from '@ionic/storage';

import { routes } from './app/app.routes';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules)),

    // 2. Configuramos Ionic Storage
    importProvidersFrom(
      IonicStorageModule.forRoot({
        name: '__mydb', // Nombre de la base de datos (puedes poner el nombre
        driverOrder: [Drivers.IndexedDB, Drivers.LocalStorage] // Orden de pre
      })
    ),
  ],
});
```

> ### ¿Qué son los Drivers?
>
> La propiedad `driverOrder` le dice a Ionic qué tecnología intentar usar primero.
> **IndexedDB** es la mejor opción para navegadores web y móviles modernos (es rápida y tiene mucha capacidad).
> **LocalStorage** es la opción de respaldo más básica.

Paso 3: ¡Listo para usar!

Con esto, la infraestructura ya está montada. Ahora cualquier servicio o componente de nuestra aplicación puede pedir (inyectar) el servicio `Storage` y empezar a guardar datos.

En la siguiente sección, crearemos un **`SettingsService`** específico para gestionar nuestros ajustes utilizando esta herramienta.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.11. Creando un SettingsService

4.11. Creando un SettingsService

Para gestionar la persistencia de los ajustes (como el modo oscuro), vamos a crear un servicio dedicado. Esto sigue el principio de responsabilidad única: no queremos "ensuciar" nuestros componentes con llamadas directas a la base de datos.

Paso 1: Generar el Servicio

Usamos la CLI de Ionic para crear el servicio en nuestra carpeta de servicios. Recuerda añadir la extensión `.service` para evitar problemas de nombres.

```
ionic generate service services/settings.service
```

Paso 2: Implementar la Lógica de Almacenamiento

Vamos a editar `src/app/services/settings.service.ts`. Aquí es donde ocurre la magia.

Necesitamos hacer tres cosas:

1. Inyectar el servicio `Storage`.
2. Crear un método `init()` para inicializar la base de datos (es obligatorio antes de usarla).
3. Crear métodos para guardar (`set`) y leer (`get`) datos.

```
// src/app/services/settings.service.ts

import { Injectable } from '@angular/core';
import { Storage } from '@ionic/storage-angular'; // 1. Importamos Storage
```

```
@Injectable({
  providedIn: 'root'
})
export class SettingsService {
  // Variable privada para guardar la instancia de la base de datos
  private _storage: Storage | null = null;

  // 2. Inyectamos el servicio Storage en el constructor
  constructor(private storage: Storage) {
    this.init(); // Iniciamos la base de datos al arrancar el servicio
  }

  /**
   * Inicializa la base de datos de Ionic Storage.
   * Es vital llamar a esto antes de intentar leer o escribir.
   */
  async init(): Promise<void> { // Añadimos el tipo de retorno explícito
    // Si ya está iniciada, no hacemos nada
    if (this._storage != null) {
      return;
    }

    // Creamos la instancia
    const storage = await this.storage.create();
    this._storage = storage;
  }

  /**
   * Guarda un valor en la base de datos asociado a una clave.
   * @param key La clave única (ej: 'modo_oscuro')
   * @param value El valor a guardar (ej: true, 'Javier', etc.)
   */
  public async set(key: string, value: any): Promise<void> {
    // Nos aseguramos de que esté iniciada
    await this.init();
    await this._storage?.set(key, value);
  }

  /**
   * Recupera un valor de la base de datos.
   * @param key La clave a buscar
   * @returns El valor guardado o null si no existe
   */
  public async get(key: string): Promise<any> {
    await this.init();
```

```
    return await this._storage?.get(key);
  }

  /**
   * Elimina un valor de la base de datos.
   */
  public async remove(key: string): Promise<void> {
    await this.init();
    await this._storage?.remove(key);
  }
}
```

Puntos Clave a Entender

- **Asincronía (`async/await`):** Todas las operaciones de base de datos son asíncronas. No ocurren instantáneamente; tardan unos milisegundos. Por eso, todos nuestros métodos deben usar `async` y devolver una `Promise`. Cuando los usemos en nuestros componentes, tendremos que usar `await` o `.then()`.
- **Tipos de Promesas:**
  - Usamos `Promise<void>` (vacío) cuando el método solo realiza una acción (como guardar o borrar) y no devuelve ningún dato.
  - Usamos `Promise<any>` cuando el método devuelve un dato (como `get`), indicando que lo que recuperamos puede ser de cualquier tipo (número, texto, objeto...).
- **Inicialización:** Ionic Storage necesita "arrancar" (abrir la conexión con SQLite o IndexedDB) antes de poder usarse. Por eso llamamos a `storage.create()`.

¡Listo! Ya tenemos un servicio robusto para guardar cualquier cosa en el dispositivo. En la siguiente sección, lo usaremos para implementar el Modo Oscuro persistente.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.12. Aplicando los Ajustes al Iniciar la App

4.12. Aplicando los Ajustes al Iniciar la App

Ya tenemos un servicio capaz de guardar y leer datos del almacenamiento del dispositivo. Ahora vamos a utilizarlo para implementar una funcionalidad real: **recordar si el usuario prefiere el Modo Oscuro**.

Paso 1: Crear la Página de Ajustes

Primero necesitamos una interfaz donde el usuario pueda cambiar esta preferencia. Generamos una nueva página:

```
ionic generate page pages/ajustes
```

En el HTML de esta página (`src/app/pages/ajustes/ajustes.page.html`), vamos a aprovechar la estructura nativa que nos da Ionic, pero añadiendo el botón para volver atrás y nuestra lista de opciones.

```
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-buttons slot="start">
      <!-- Botón para volver a la Home -->
      <ion-back-button defaultHref="/home"></ion-back-button>
    </ion-buttons>
    <ion-title>Ajustes</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true">
```

```
    <!-- Header colapsable para iOS -->
    <ion-header collapse="condense">
      <ion-toolbar>
        <ion-title size="large">Ajustes</ion-title>
      </ion-toolbar>
    </ion-header>

    <!-- Nuestra lista de configuración -->
    <ion-list>
      <ion-list-header>Apariencia</ion-list-header>
      <ion-item>
        <ion-label>Modo Oscuro</ion-label>
        <!-- Enlazamos el toggle con una variable y escuchamos el cambio -->
        <ion-toggle [(ngModel)]="modoOscuro" (ionChange)="cambiarModoOscuro()"><
      </ion-item>
    </ion-list>
  </ion-content>
```

Paso 2: Conectar la Página con el Servicio

Ahora vamos a la lógica de la página (`src/app/pages/ajustes/ajustes.page.ts`). Aquí inyectamos nuestro `SettingsService` y programamos la lógica para guardar el cambio. Asegúrate de importar todos los componentes que usas en el HTML.

```
// src/app/pages/ajustes/ajustes.page.ts

import { Component, OnInit } from '@angular/core';
import { SettingsService } from 'src/app/services/settings.service';
import { FormsModule } from '@angular/forms';
import { IonContent, IonHeader, IonTitle, IonToolbar, IonList, IonListHeader,

@Component({
  selector: 'app-ajustes',
  templateUrl: './ajustes.page.html',
  styleUrls: ['./ajustes.page.scss'],
  standalone: true,
  imports: [
    IonContent, IonHeader, IonTitle, IonToolbar,
    IonButtons, IonBackButton,
    IonList, IonListHeader, IonItem, IonLabel, IonToggle,
    FormsModule
  ]
```

```
})
export class AjustesPage implements OnInit {

  modoOscuro: boolean = false; // Valor por defecto

  constructor(private settingsService: SettingsService) { }

  // ¡IMPORTANTE! Añadimos 'async' para poder usar 'await' dentro
  async ngOnInit() {
    // Al entrar, cargamos el valor guardado
    // Si no existe (es la primera vez), settingsService.get devuelve null,
    // así que usamos '|| false' para que sea false por defecto.
    this.modoOscuro = await this.settingsService.get('modo_oscuro') || false;

    // Aplicamos el tema inmediatamente al entrar por si acaso
    this.aplicarTema(this.modoOscuro);
  }

  // También debe ser async porque settingsService.set devuelve una promesa
  async cambiarModoOscuro() {
    // 1. Guardamos el nuevo valor en la base de datos
    await this.settingsService.set('modo_oscuro', this.modoOscuro);

    // 2. Aplicamos el cambio visualmente
    this.aplicarTema(this.modoOscuro);
  }

  aplicarTema(esOscuro: boolean) {
    // Añadimos o quitamos la clase 'dark' al body del documento
    // Esto activa los estilos que definiremos en variables.scss
    document.body.classList.toggle('dark', esOscuro);
  }
}
```

> ### Consejo Pro: Unificar Estilos con Mixins (SCSS)
>
> Para no tener que copiar y pegar los mismos colores dos veces (una para el modo automático y otra para el manual), podemos usar un **Mixin** de Sass en `variables.scss`.

```
/* src/theme/variables.scss */
```

```scss
/* 1. Definimos un "grupo" de estilos reutilizable (mixin) */
@mixin dark-theme-variables {
  --ion-color-primary: #428cff;
  --ion-color-primary-rgb: 66,140,255;
  --ion-color-primary-contrast: #ffffff;

  --ion-background-color: #121212;
  --ion-text-color: #ffffff;
  /* ... resto de variables ... */
}

/* 2. Lo aplicamos para detección automática (Sistema) */
@media (prefers-color-scheme: dark) {
  body {
    @include dark-theme-variables;
  }
}

/* 3. Lo aplicamos para activación manual (Clase .dark) */
body.dark {
  @include dark-theme-variables;
}
```

Con esto, defines los colores en un solo sitio y funcionan tanto si el usuario cambia el ajuste en su móvil como si pulsa el botón en tu app.

Paso 3: Cargar el Ajuste al Iniciar la App

Lo que hemos hecho hasta ahora funciona, pero tiene un fallo: si cierras la app y la vuelves a abrir, el modo oscuro no se aplicará hasta que visites la página de ajustes. ¡Queremos que se aplique nada más abrir la app!

Para eso, debemos ir al componente raíz de toda la aplicación: **`src/app/app.component.ts`**.

```typescript
// src/app/app.component.ts

import { Component } from '@angular/core';
import { IonApp, IonRouterOutlet } from '@ionic/angular/standalone';
import { SettingsService } from './services/settings.service';
```

```
@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  imports: [IonApp, IonRouterOutlet],
})
export class AppComponent {

  constructor(private settingsService: SettingsService) {}

  // Angular ejecutará este método automáticamente al iniciar,
  // aunque no implementemos la interfaz OnInit explícitamente.
  // ¡IMPORTANTE! Añadimos 'async' para esperar a la base de datos.
  async ngOnInit() {
    await this.cargarPreferencias();
  }

  async cargarPreferencias() {
    // Leemos el ajuste guardado
    const modoOscuro = await this.settingsService.get('modo_oscuro');

    // Si era true, activamos el modo oscuro inmediatamente
    if (modoOscuro) {
      document.body.classList.add('dark');
    }
  }
}
```

¿Qué es `ngOnInit`?

`ngOnInit` es un método especial del ciclo de vida de Angular. Angular lo llama automáticamente **una sola vez**, justo después de crear el componente y configurar sus propiedades. Aunque no es obligatorio poner `implements OnInit` en la clase, Angular buscará este método y lo ejecutará si existe.

¿Por qué no usar el `constructor`?

El `constructor` debe usarse solo para la inyección de dependencias (pedir servicios). Si intentamos hacer lógica compleja o llamadas a bases de datos en el constructor, podríamos bloquear la creación del componente o encontrarnos con que la vista aún no

> está lista. Por eso, cualquier lógica de inicialización (como cargar datos) siempre debe ir en `ngOnInit`.

> ### Concepto Clave: `ngOnInit` Asíncrono
>
> Por defecto, el método `ngOnInit()` se genera sin la palabra clave `async`. Sin embargo, como necesitamos leer datos de la base de datos (una operación que tarda un tiempo), debemos usar `await` para esperar el resultado.
>
> **Recordatorio de la regla de oro:**
>
> - **`await` (esperar):** Detiene la ejecución del código en esa línea hasta que la promesa (la operación de base de datos) termine y devuelva el valor.
> - **`async` (asíncrono):** Para poder usar `await` dentro de una función, **es obligatorio** marcar esa función como `async`.
>
> Por eso hemos añadido la firma: `async ngOnInit() { ... }`.

Paso 4: Añadir Acceso desde la Home

Finalmente, necesitamos una forma de llegar a la página de ajustes. Vamos a añadir un botón con icono de engranaje en la barra de herramientas de `home.page.html`.

4.1. Importar Dependencias en `home.page.ts`

Para que el enlace funcione, asegúrate de importar `RouterLink`, `IonButtons` e `IonIcon`.

```
// src/app/home/home.page.ts
import { RouterLink } from '@angular/router';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonList, IonItem, IonIn
import { addIcons } from 'ionicons';
import { settingsOutline } from 'ionicons/icons';

@Component({
  // ...
  imports: [
    // ... otros imports ...
    RouterLink,
    IonButtons, // <--- ¡Imprescindible!
```

```
    IonIcon     // <--- ¡Imprescindible!
  ]
})
export class HomePage {
  constructor(/*...*/) {
    // Registramos el icono de ajustes
    addIcons({ settingsOutline });
  }
  // ...
}
```

4.2. Añadir el Botón en `home.page.html`

Asegúrate de que `<ion-buttons>` esté fuera de `<ion-title>`. Deben ser "hermanos".

```
<ion-header [translucent]="true">
  <ion-toolbar color="primary">
    <ion-title>Mis Tareas</ion-title>

    <!-- CORRECTO: ion-buttons está fuera de ion-title -->
    <ion-buttons slot="end">
      <ion-button routerLink="/ajustes">
        <ion-icon slot="icon-only" name="settings-outline"></ion-icon>
      </ion-button>
    </ion-buttons>

  </ion-toolbar>
</ion-header>
```

¡Y listo! Ahora tu aplicación tiene memoria y un sistema de preferencias completo. Da igual cuántas veces la cierres o reinicies; si el usuario eligió el Modo Oscuro, Ionic Storage lo recordará y tu `AppComponent` lo aplicará al instante.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.13. Reto: App Multi-página y con Ajustes Persistentes

4.13. Reto: App Multi-página y con Ajustes Persistentes

En esta unidad hemos dado un salto gigante en la arquitectura de nuestra aplicación. Hemos pasado de tener todo el código mezclado en una sola página a tener una estructura profesional con **Servicios**, **Navegación** y **Persistencia**.

El objetivo de este reto es aplicar estos tres pilares a tu proyecto personal para convertirlo en una aplicación robusta y navegable.

> #### ✅ Reto Básico (Requisitos Mínimos para Aprobar)
>
> Para superar esta unidad, debes refactorizar tu aplicación actual cumpliendo los siguientes puntos:
>
> 1. **Arquitectura de Servicios:** Debes eliminar el array de datos (tareas, productos, etc.) de tu `HomePage`. Crea un servicio (ej: `TaskService`, `ProductosService`) y mueve allí tanto los datos como la lógica para añadir nuevos elementos. Tu `HomePage` debe inyectar este servicio para funcionar.
> 2. **Persistencia de Ajustes:** Implementa **Ionic Storage** y crea un `SettingsService`. Crea una nueva página de "Ajustes" donde el usuario pueda activar/desactivar el **Modo Oscuro**. Esta preferencia debe guardarse y aplicarse automáticamente al reiniciar la app (como vimos en el 4.12).
> 3. **Página de Detalle:** Crea una nueva página llamada `DetallePage` (o nombre acorde a tu tema). Configura el enrutador (`app.routes.ts`) para que acepte un parámetro `id` (ej: `path: 'detalle/:id'`).
> 4. **Navegación:** Haz que los elementos de tu lista principal sean clicables. Al pulsar uno, la aplicación debe navegar a la página de detalle pasando el ID correspondiente en la URL.

5. **Mostrar Datos Completos y Diseño:** En la página de detalle, usa `ActivatedRoute` para leer el ID y obtener el objeto del servicio. Debes mostrar **todos los datos disponibles** de ese objeto (título, descripción, fechas, estado, imágenes, etc.). No basta con volcarlos de cualquier manera; debes aplicar un **diseño adecuado y atractivo** utilizando componentes de Ionic (`ion-card`, `ion-list`, `ion-badge`, `ion-img`, etc.) para organizar la información visualmente.

🚀 Retos para la Máxima Calificación (Bien / Excelente)

Para optar a las notas más altas, debes implementar funcionalidades avanzadas de personalización y navegación:

- **Mejora 1 (Personalización):** En la página de ajustes, añade un campo de texto para que el usuario guarde su **Nombre** (usando Ionic Storage). Haz que la `HomePage` muestre un saludo personalizado (ej: "Hola, Javier") leyendo ese dato al iniciar.
- **Mejora 2 (Borrado con Navegación):** Añade un botón "Eliminar" en la página de **Detalle**. Al pulsarlo, debe llamar al servicio para borrar ese elemento y, automáticamente, redirigir al usuario de vuelta a la lista (`/home`) usando `this.router.navigate()`.
- **Mejora 3 (UX - Back Button):** Asegúrate de que tanto la página de detalle como la de ajustes tengan el botón `<ion-back-button>` en la cabecera para poder volver atrás cómodamente, respetando la navegación nativa.
- **Mejora 4 (Confirmación de Seguridad):** Mejora la función de borrado para que no sea inmediata. Al pulsar "Eliminar", debe mostrarse primero un `<ion-alert>` pidiendo confirmación. El borrado y la redirección solo deben ocurrir si el usuario pulsa "Aceptar".
- **Mejora 5 (Manejo de Errores):** Implementa lógica en la página de Detalle para detectar si el ID pasado por la URL no corresponde a ningún objeto existente (por ejemplo, si se escribe la URL a mano). En ese caso, la app debe redirigir automáticamente a la `HomePage` y mostrar un `<ion-toast>` con el mensaje "Elemento no encontrado".

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

4.14. Recursos Adicionales

4.14. Recursos Adicionales

En esta unidad hemos cubierto una gran cantidad de conceptos fundamentales de la arquitectura de aplicaciones con Ionic y Angular. Si quieres profundizar en alguno de ellos o necesitas consultar la referencia técnica para resolver dudas específicas, aquí tienes una selección de los mejores recursos oficiales.

📚 Documentación Oficial de Angular

Angular es el motor que mueve nuestra aplicación. Entender sus conceptos base es clave para ser un buen desarrollador de Ionic.

- Enrutamiento y Navegación (Routing) <https://angular.dev/guide/routing>
  La guía definitiva sobre cómo funciona el router, cómo configurar rutas hijas, guardias de navegación (guards) y estrategias de carga.
- Inyección de Dependencias (Dependency Injection) <https://angular.dev/guide/di>
  Explicación profunda sobre qué son los servicios, los inyectores y los proveedores. Vital para entender la "magia" de los servicios.
- API de ActivatedRoute <https://angular.dev/api/router/ActivatedRoute>
  Referencia técnica de la clase que usamos para leer los parámetros de la URL (`snapshot.paramMap`).

📱 Documentación de Ionic Framework

Recursos específicos sobre los componentes visuales y las herramientas que Ionic añade sobre Angular.

- Navegación en Ionic y Angular <https://ionicframework.com/docs/angular/navigation>
  Cómo Ionic integra el Router de Angular con sus propias animaciones de transición nativas y el manejo del historial (pila de navegación).
- Ciclo de Vida de las Páginas <https://ionicframework.com/docs/angular/lifecycle>
  Entender cuándo usar `ngOnInit` vs `ionViewWillEnter` es crucial. Esta guía explica los eventos propios de Ionic.

- ion-modal API <https://ionicframework.com/docs/api/modal>
  Todo lo que puedes hacer con los modales: pasar datos, estilos personalizados, breakpoints para hojas inferiores (bottom sheets), etc.

💾 Persistencia de Datos

- Ionic Storage (GitHub) <https://github.com/ionic-team/ionic-storage>
  El repositorio oficial de la librería que hemos usado. Contiene ejemplos de configuración avanzada y uso de diferentes drivers.
- Promesas en JavaScript (MDN) <https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise>
  Si te has sentido un poco perdido con `async` y `await`, esta es la mejor guía para entender cómo funciona la asincronía en JavaScript moderno.

🛠️ Herramientas Útiles

- Angular DevTools <https://angular.dev/tools/devtools>
  Una extensión imprescindible para depurar aplicaciones Angular. Te permite ver el árbol de componentes, el estado de los servicios y el router en tiempo real.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0** <http://creativecommons.org/licenses/by-sa/4.0/>